Primality test

A **primality test** is an algorithm for determining whether an input number is prime. Among other fields of mathematics, it is used for cryptography. Unlike integer factorization, primality tests do not generally give prime factors, only stating whether the input number is prime or not. Factorization is thought to be a computationally difficult problem, whereas primality testing is comparatively easy (its running time is polynomial in the size of the input). Some primality tests prove that a number is prime, while others like Miller–Rabin prove that a number is composite. Therefore, the latter might more accurately be called *compositeness tests* instead of primality tests.

Simple methods

The simplest primality test is *trial division*: given an input number, n, check whether it is divisible by any prime number between 2 and $\sqrt{n}$ (i.e., whether the division leaves no remainder). If so, then n is composite. Otherwise, it is prime.[1] For any divisor $p \geq \sqrt{n}$, there must be another divisor $n/p \leq \sqrt{n}$, and a prime divisor q of n/p, and therefore looking for prime divisors at most $\sqrt{n}$ is sufficient.

For example, consider the number 100, whose divisors are these numbers:

$$1, 2, 4, 5, 10, 20, 25, 50, 100.$$

When all possible divisors up to n are tested, some divisors will be discovered *twice*. To observe this, consider the list of divisor pairs of 100:

$$1 \times 100,\ 2 \times 50,\ 4 \times 25,\ 5 \times 20,\ 10 \times 10,\ 20 \times 5,\ 25 \times 4,\ 50 \times 2,\ 100 \times 1.$$

Products past 10×10 are the reverse of products that appeared earlier. For example, 5×20 and 20×5 are the reverse of each other. Further, that of the two divisors, $5 \leq \sqrt{100} = 10$ and $20 \geq \sqrt{100} = 10$. This observation generalizes to all n: all divisor pairs of n contain a divisor less than or equal to $\sqrt{n}$, so the algorithm need only search for divisors less than or equal to $\sqrt{n}$ to guarantee detection of all divisor pairs.[1]

Also, 2 is a prime dividing 100, which immediately proves that 100 is not prime. Every positive integer except 1 is divisible by at least one prime number by the Fundamental Theorem of Arithmetic. Therefore the algorithm need only search for *prime* divisors less than or equal to $\sqrt{n}$.

For another example, consider how this algorithm determines the primality of 17. One has $4 < \sqrt{17} < 5$, and the only primes $\leq \sqrt{17}$ are 2 and 3. Neither divides 17, proving that 17 is prime. For a last example, consider 221. One has $14 < \sqrt{221} < 15$, and the primes $\leq \sqrt{221}$ are 2, 3, 5, 7, 11, and 13. Upon checking each, one discovers that $221/13 = 17$, proving that 221 is not prime.

In cases where it is not feasible to compute the list of primes $\leq \sqrt{n}$, it is also possible to simply (and slowly) check all numbers between 2 and $\sqrt{n}$ for divisors. A simple improvement is to test divisibility by 2 and by just the odd numbers between 3 and $\sqrt{n}$, since divisibility by an even number implies divisibility by 2.

This method can be improved further. Observe that all primes greater than 5 are of the form $6k+i$ for a nonnegative integer k and $i \in \{1,5\}$. Indeed, every integer is of the form $6k+i$ for a positive integer k and $i \in \{0,1,2,3,4,5\}$. Since 2 divides $6k, 6k+2$, and $6k+4$, and 3 divides $6k$ and $6k+3$, the only possible remainders mod 6 for a prime greater than 3 are 1 and 5. So, a more efficient primality test for n is to test whether n is divisible by 2 or 3, then to check through all numbers of the form $6k+1$ and $6k+5$ which are $\leq \sqrt{n}$. This is almost three times as fast as testing all numbers up to $\sqrt{n}$.

Generalizing further, all primes greater than $c\#$ (c primorial) are of the form $c\# \cdot k + i$ for i, k positive integers, $0 \leq i < c\#$, and i coprime to $c\#$. For example, consider $6\# = 2 \cdot 3 \cdot 5 = 30$. All integers are of the form $30k+i$ for i, k integers with $0 \leq i < 30$. Now, 2 divides $0, 2, 4, \ldots, 28$, 3 divides $0, 3, 6, \ldots, 27$, and 5 divides $0, 5, 10, \ldots, 25$. Thus all prime numbers greater than 30 are of the form $30k+i$ for $i \in \{1, 7, 11, 13, 17, 19, 23, 29\}$. Of course, not all numbers of the form $c\# \cdot k + i$ with i coprime to $c\#$ are prime. For example, $19 \cdot 23 = 437 = 210 \cdot 2 + 17 = 2 \cdot 7\# + 17$ is not prime, even though 17 is coprime to $7\# = 2 \cdot 3 \cdot 5 \cdot 7$.

As c grows, the fraction of coprime remainders to remainders decreases, and so the time to test n decreases (though it still necessary to check for divisibility by all primes that are less than c). Observations analogous to the preceding can be applied recursively, giving the Sieve of Eratosthenes.

One way to speed up these methods (and all the others mentioned below) is to pre-compute and store a list of all primes up to a certain bound, such as all primes up to 200. (Such a list can be computed with the Sieve of Eratosthenes or by an algorithm that tests each incremental m against all known primes $\leq \sqrt{m}$). Then, before testing n for primality with a large-scale method, n can first be checked for divisibility by any prime from the list. If it is divisible by any of those numbers then it is composite, and any further tests can be skipped.

A simple but inefficient primality test uses Wilson's theorem, which states that p is prime if and only if:

$$(p-1)! \equiv -1 \pmod{p}$$

Although this method requires about p modular multiplications,[2] rendering it impractical, theorems about primes and modular residues form the basis of many more practical methods.

Heuristic tests

These are tests that seem to work well in practice, but are unproven and therefore are not, technically speaking, algorithms at all. The Fermat primality test and the Fibonacci test are simple examples, and they are effective when combined. John Selfridge has conjectured that if p is an odd number, and $p \equiv \pm 2 \pmod 5$, then p will be prime if both of the following hold:

- $2^{p-1} \equiv 1 \pmod{p}$,
- $f_{p+1} \equiv 0 \pmod{p}$,

where f_k is the k-th Fibonacci number. The first condition is the Fermat primality test using base 2.

In general, if $p \equiv a \pmod{x^2+4}$, where a is a quadratic non-residue $\pmod{x^2+4}$ then p should be prime if the following conditions hold:

- $2^{p-1} \equiv 1 \pmod{p}$,

- $f(x)_{p+1} \equiv 0 \pmod{p}$,

$f(x)_k$ is the k-th Fibonacci polynomial at x.

Selfridge, Pomerance and Wagstaff together offered $620 for a counterexample or proof that there is none,[3] with the prize now due from the Number Theory Foundation.

Probabilistic tests

Probabilistic tests are more rigorous than heuristics in that they provide provable bounds on the probability of being fooled by a composite number. Multiple popular primality tests are probabilistic tests. These tests use, apart from the tested number n, some other numbers a which are chosen at random from some sample space; the usual randomized primality tests never report a prime number as composite, but it is possible for a composite number to be reported as prime. The probability of error can be reduced by repeating the test with several independently chosen values of a; for two commonly used tests, for *any* composite n at least half the a's detect n's compositeness, so k repetitions reduce the error probability to at most 2^{-k}, which can be made arbitrarily small by increasing k.

The basic structure of randomized primality tests is as follows:

1. Randomly pick a number a.
2. Check equality (corresponding to the chosen test) involving a and the given number n. If the equality fails to hold true, then n is a composite number and a is a *witness* for the compositeness, and the test stops.
3. Get back to the step one until the required accuracy is reached.

After one or more iterations, if n is not found to be a composite number, then it can be declared probably prime.

Fermat primality test

The simplest probabilistic primality test is the Fermat primality test (actually a compositeness test). It works as follows:

> Given an integer n, choose some integer a coprime to n and calculate a^{n-1} modulo n. If the result is different from 1, then n is composite. If it is 1, then n may be prime.

If a^{n-1} (modulo n) is 1 but n is not prime, then n is called a pseudoprime to base a. In practice, if a^{n-1} (modulo n) is 1, then n is usually prime. But here is a counterexample: if $n = 341$ and $a = 2$, then

$$2^{340} \equiv 1 \pmod{341}$$

even though $341 = 11 \cdot 31$ is composite. In fact, 341 is the smallest pseudoprime base 2 (see Figure 1 of [4]).

There are only 21853 pseudoprimes base 2 that are less than 2.5×10^{10} (see page 1005 of [4]). This means that, for n up to 2.5×10^{10}, if 2^{n-1} (modulo n) equals 1, then n is prime, unless n is one of these 21853 pseudoprimes.

Some composite numbers (Carmichael numbers) have the property that a^{n-1} is 1 (modulo n) for every a that is coprime to n. The smallest example is $n = 561 = 3 \cdot 11 \cdot 17$, for which a^{560} is 1 (modulo 561) for all a coprime to 561. Nevertheless, the Fermat test is often used if a rapid screening of numbers is needed, for instance in the key generation phase of the RSA public key cryptographic algorithm.

Miller–Rabin and Solovay–Strassen primality test

The Miller–Rabin primality test and Solovay–Strassen primality test are more sophisticated variants, which detect all composites (once again, this means: for *every* composite number n, at least 3/4 (Miller–Rabin) or 1/2 (Solovay–Strassen) of numbers a are witnesses of compositeness of n). These are also compositeness tests.

The Miller–Rabin primality test works as follows: Given an integer n, choose some positive integer $a < n$. Let $2^s d = n - 1$, where d is odd. If

$$a^d \not\equiv \pm 1 \pmod{n}$$

and

$$a^{2^r d} \not\equiv -1 \pmod{n} \text{ for all } 0 \le r \le s-1,$$

then n is composite and a is a witness for the compositeness. Otherwise, n may or may not be prime. The Miller–Rabin test is a strong probable prime test (see PSW[4] page 1004).

The Solovay–Strassen primality test uses another equality: Given an odd number n, choose some integer $a < n$, if

$$a^{(n-1)/2} \not\equiv \left(\frac{a}{n}\right) \pmod{n}, \text{ where } \left(\frac{a}{n}\right) \text{ is the Jacobi symbol,}$$

then n is composite and a is a witness for the compositeness. Otherwise, n may or may not be prime. The Solovay–Strassen test is an Euler probable prime test (see PSW[4] page 1003).

For each individual value of a, the Solovay–Strassen test is weaker than the Miller–Rabin test. For example, if $n = 1905$ and $a = 2$, then the Miller-Rabin test shows that n is composite, but the Solovay–Strassen test does not. This is because 1905 is an Euler pseudoprime base 2 but not a strong pseudoprime base 2 (this is illustrated in Figure 1 of PSW[4]).

Frobenius primality test

The Miller–Rabin and the Solovay–Strassen primality tests are simple and are much faster than other general primality tests. One method of improving efficiency further in some cases is the Frobenius pseudoprimality test; a round of this test takes about three times as long as a round of Miller–Rabin, but achieves a probability bound comparable to seven rounds of Miller–Rabin.

The Frobenius test is a generalization of the Lucas probable prime test.

Baillie–PSW primality test

The Baillie–PSW primality test is a probabilistic primality test that combines a Fermat or Miller–Rabin

test with a Lucas probable prime test to get a primality test that has no known counterexamples. That is, there are no known composite n for which this test reports that n is probably prime.[5][6] It has been shown that there are no counterexamples for $n < 2^{64}$.

Other tests

Leonard Adleman and Ming-Deh Huang presented an errorless (but expected polynomial-time) variant of the elliptic curve primality test. Unlike the other probabilistic tests, this algorithm produces a primality certificate, and thus can be used to prove that a number is prime.[7] The algorithm is prohibitively slow in practice.

If quantum computers were available, primality could be tested asymptotically faster than by using classical computers. A combination of Shor's algorithm, an integer factorization method, with the Pocklington primality test could solve the problem in $O((\log n)^3 (\log\log n)^2 \log\log\log n)$.[8]

Fast deterministic tests

Near the beginning of the 20th century, it was shown that a corollary of Fermat's little theorem could be used to test for primality.[9] This resulted in the Pocklington primality test.[10] However, as this test requires a partial factorization of $n - 1$ the running time was still quite slow in the worst case. The first deterministic primality test significantly faster than the naive methods was the cyclotomy test; its runtime can be proven to be $O((\log n)^{c \log\log\log n})$, where n is the number to test for primality and c is a constant independent of n. A number of further improvements were made, but none could be proven to have polynomial running time. (Running time is measured in terms of the size of the input, which in this case is ~ $\log n$, that being the number of bits needed to represent the number n.) The elliptic curve primality test can be proven to run in $O((\log n)^6)$, if some conjectures on analytic number theory are true. Similarly, under the generalized Riemann hypothesis (which Miller, confusingly, calls the "extended Riemann hypothesis"), the deterministic Miller's test, which forms the basis of the probabilistic Miller–Rabin test, can be proved to run in $\tilde{O}((\log n)^4)$.[11] In practice, this algorithm is slower than the other two for sizes of numbers that can be dealt with at all. Because the implementation of these two methods is rather difficult and creates a risk of programming errors, slower but simpler tests are often preferred.

In 2002, the first provably unconditional deterministic polynomial time test for primality was invented by Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. The AKS primality test runs in $\tilde{O}((\log n)^{12})$ (improved to $\tilde{O}((\log n)^{7.5})$[12] in the published revision of their paper), which can be further reduced to $\tilde{O}((\log n)^6)$ if the Sophie Germain conjecture is true.[13] Subsequently, Lenstra and Pomerance presented a version of the test which runs in time $\tilde{O}((\log n)^6)$ unconditionally.[14]

Agrawal, Kayal and Saxena suggest a variant of their algorithm which would run in $\tilde{O}((\log n)^3)$ if Agrawal's conjecture is true; however, a heuristic argument by Hendrik Lenstra and Carl Pomerance suggests that it is probably false.[12] A modified version of the Agrawal's conjecture, the Agrawal–Popovych conjecture,[15] may still be true.

Complexity

In computational complexity theory, the formal language corresponding to the prime numbers is denoted as PRIMES. It is easy to show that PRIMES is in **Co-NP**: its complement COMPOSITES is in **NP** because one can decide compositeness by nondeterministically guessing a factor.

In 1975, Vaughan Pratt showed that there existed a certificate for primality that was checkable in polynomial time, and thus that PRIMES was in NP, and therefore in $\mathbf{NP} \cap \mathbf{coNP}$. See primality certificate for details.

The subsequent discovery of the Solovay–Strassen and Miller–Rabin algorithms put PRIMES in coRP. In 1992, the Adleman–Huang algorithm[7] reduced the complexity to $\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{coRP}$, which superseded Pratt's result.

The Adleman–Pomerance–Rumely primality test from 1983 put PRIMES in **QP** (quasi-polynomial time), which is not known to be comparable with the classes mentioned above.

Because of its tractability in practice, polynomial-time algorithms assuming the Riemann hypothesis, and other similar evidence, it was long suspected but not proven that primality could be solved in polynomial time. The existence of the AKS primality test finally settled this long-standing question and placed PRIMES in **P**. However, PRIMES is not known to be P-complete, and it is not known whether it lies in classes lying inside **P** such as NC or L. It is known that PRIMES is not in AC^0.[16]

Number-theoretic methods

Certain number-theoretic methods exist for testing whether a number is prime, such as the Lucas test and Proth's test. These tests typically require factorization of $n + 1$, $n - 1$, or a similar quantity, which means that they are not useful for general-purpose primality testing, but they are often quite powerful when the tested number n is known to have a special form.

The Lucas test relies on the fact that the multiplicative order of a number a modulo n is $n - 1$ for a prime n when a is a primitive root modulo n. If we can show a is primitive for n, we can show n is prime.

References

1. Riesel (1994) pp.2-3
2. Barrus, Mike; Clark, W. Edwin (2021-09-05). "Primality Tests" (https://math.libretexts.org/Bookshelves/Combinatorics_and_Discrete_Mathematics/Elementary_Number_Theory_(Barrus_and_Clark)/01:_Chapters/1.25:_Primality_Tests). *Elementary Number Theory* (https://math.libretexts.org/Bookshelves/Combinatorics_and_Discrete_Mathematics/Elementary_Number_Theory_(Barrus_and_Clark)). Mathematics LibreTexts. Retrieved 2025-03-14.
3. Guy, Richard (1994). *Unsolved problems in number theory*. Springer-Verlag: New York. p. 28. ISBN 0387942890.

4. Pomerance, Carl; Selfridge, John L.; Wagstaff, Samuel S. Jr. (July 1980). "The pseudoprimes to $25 \cdot 10^9$" (https://www.math.dartmouth.edu/~carlp/PDF/paper25.pdf) (PDF). *Mathematics of Computation*. **35** (151): 1003–1026. doi:10.1090/S0025-5718-1980-0572872-7 (https://doi.org/10.1090%2FS0025-5718-1980-0572872-7).
5. Baillie, Robert; Wagstaff, Samuel S. Jr. (October 1980). "Lucas Pseudoprimes" (https://mpqs.free.fr/LucasPseudoprimes.pdf) (PDF). *Mathematics of Computation*. **35** (152): 1391–1417. doi:10.1090/S0025-5718-1980-0583518-6 (https://doi.org/10.1090%2FS0025-5718-1980-0583518-6). MR 0583518 (https://mathscinet.ams.org/mathscinet-getitem?mr=0583518).
6. Baillie, Robert; Fiori, Andrew; Wagstaff, Samuel S. Jr. (July 2021). "Strengthening the Baillie-PSW Primality Test". *Mathematics of Computation*. **90** (330): 1931–1955. arXiv:2006.14425 (https://arxiv.org/abs/2006.14425). doi:10.1090/mcom/3616 (https://doi.org/10.1090%2Fmcom%2F3616). S2CID 220055722 (https://api.semanticscholar.org/CorpusID:220055722).
7. Adleman, Leonard M.; Huang, Ming-Deh (1992). *Primality testing and Abelian varieties over finite field*. Lecture notes in mathematics. Vol. 1512. Springer-Verlag. ISBN 3-540-55308-8.
8. Chau, H. F.; Lo, H.-K. (1995). "Primality Test Via Quantum Factorization". arXiv:quant-ph/9508005 (https://arxiv.org/abs/quant-ph/9508005).
9. Pocklington, H. C. (1914). "The determination of the prime or composite nature of large numbers by Fermat's theorem". *Cambr. Phil. Soc. Proc*. **18**: 29–30. JFM 45.1250.02 (https://zbmath.org/?format=complete&q=an:45.1250.02).
10. Weisstein, Eric W. "Pocklington's Theorem" (https://mathworld.wolfram.com/PocklingtonsTheorem.html). *MathWorld*.
11. Gary L. Miller (1976). "Riemann's Hypothesis and Tests for Primality" (https://doi.org/10.1016%2FS0022-0000%2876%2980043-8). *Journal of Computer and System Sciences*. **13** (3): 300–317. doi:10.1016/S0022-0000(76)80043-8 (https://doi.org/10.1016%2FS0022-0000%2876%2980043-8).
12. Agrawal, Manindra; Kayal, Neeraj; Saxena, Nitin (2004). "Primes is in P" (http://annals.math.princeton.edu/wp-content/uploads/annals-v160-n2-p12.pdf) (PDF). *Annals of Mathematics*. **160** (2): 781–793. doi:10.4007/annals.2004.160.781 (https://doi.org/10.4007%2Fannals.2004.160.781).
13. Agrawal, Manindra; Kayal, Neeraj; Saxena, Nitin (2004). "PRIMES is in P" (http://www.cse.iitk.ac.in/users/manindra/algebra/primality_v6.pdf) (PDF). *Annals of Mathematics*. **160** (2): 781–793. doi:10.4007/annals.2004.160.781 (https://doi.org/10.4007%2Fannals.2004.160.781).
14. Carl Pomerance & Hendrik W. Lenstra (July 20, 2005). "Primality testing with Gaussian periods" (http://www.math.dartmouth.edu/~carlp/PDF/complexity12.pdf) (PDF).
15. Popovych, Roman (December 30, 2008). "A note on Agrawal conjecture" (http://eprint.iacr.org/2009/008.pdf) (PDF).
16. Allender, Eric; Saks, Michael; Shparlinski, Igor (2001). "A Lower Bound for Primality" (https://doi.org/10.1006/jcss.2000.1725). *Journal of Computer and System Sciences*. **62** (2): 356–366. doi:10.1006/jcss.2000.1725 (https://doi.org/10.1006%2Fjcss.2000.1725).

Sources

- Crandall, Richard; Pomerance, Carl (2005). *Prime Numbers: A Computational Perspective* (2nd ed.). Springer. ISBN 0-387-25282-7. Chapter 3: Recognizing Primes and Composites, pp. 109–158. Chapter 4: Primality Proving, pp. 159–190. Section 7.6: Elliptic curve primality proving (ECPP), pp. 334–340.
- Knuth, Donald (1997). "section 4.5.4". *The Art of Computer Programming*. Vol. 2: *Seminumerical Algorithms* (3rd ed.). Addison–Wesley. pp. 391–396. ISBN 0-201-89684-2.
- Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2001). "Section 31.8: Primality testing". *Introduction to Algorithms* (Second ed.). MIT Press, McGraw–Hill. pp. 887–896. ISBN 0-262-03293-7.
- Papadimitriou, Christos H. (1993). "Section 10.2: Primality". *Computational Complexity* (https://archive.org/details/computationalcom00papa) (1st ed.). Addison Wesley. pp. 222 (https://archive.org/details/computationalcom00papa/page/n230)–227. ISBN 0-201-53082-1. Zbl 0833.68049 (https://zbmath.org/?format=complete&q=an:0833.68049).
- Riesel, Hans (1994). *Prime Numbers and Computer Methods for Factorization*. Progress in Mathematics. Vol. 126 (second ed.). Boston, Massachusetts: Birkhäuser. ISBN 0-8176-3743-5. Zbl 0821.11001 (https://zbmath.org/?format=complete&q=an:0821.11001).

External links

- Deprecated link at archive.today (archived 2012-12-20) – Implementation of the Solovay-Strassen primality test in Maple
- Distinguishing prime numbers from composite numbers, by D.J. Bernstein (cr.yp.to) (http://cr.yp.to/primetests.html)
- The Prime Pages (primes.utm.edu) (http://primes.utm.edu/)
- Lucas Primality Test with Factored $N - 1$ (MathPages.com) (http://webarchive.loc.gov/all/20100806075947/http://mathpages.com/home/kmath473.htm) at the Library of Congress Web Archives (archived 2010-08-06)
- PRIMABOINCA (http://www.primaboinca.com/) is a research project that uses Internet-connected computers to search for a counterexample to some conjectures. The first conjecture (Agrawal's conjecture) was the basis for the formulation of the first deterministic prime test algorithm in polynomial time (AKS algorithm).

Retrieved from "https://en.wikipedia.org/w/index.php?title=Primality_test&oldid=1321400226"